BÁO CÁO TỔNG HỢP DỰ ÁN CUỐI KỲ

HỆ THỐNG THƯ VIỆN ĐIỆN TỬ PHÂN TÁN (E-LIBRARY)

Nhóm Sinh viên:
Nguyễn Tiến Đạt - 835151465
Đặng Trung Kiên - 835151466
Nguyễn Diệp Linh - 835151467

Ngày 05 tháng 01 năm 2026

Link Dự án:

https://bitbucket.org/dtn__ntd/elib/src/dtn_elib/

Mục lục

1 Giới thiệu & Mục tiêu Hệ thống **4**
1.1 Mục tiêu Chính 4
1.2 Phạm vi Dự án (Scope) 4

2 Công nghệ & Tổ chức Mã nguồn **4**
2.1 Công nghệ Sử dụng 4
2.2 Cấu trúc Dự án 4
2.3 Tổ chức Docker 5

3 Kiến trúc Phân tán (Infrastructure) **5**
3.1 Sơ đồ Hạ tầng & Các Node Phân tán 5
3.2 Chứng minh Phân tải (Sharding Verification) 7
3.3 Nguyên lý Thiết kế 7

4 Thiết kế Dữ liệu & API **7**
4.1 Lược đồ Quan hệ (ERD) 7
4.2 API Endpoints 7
4.3 Chi tiết Implement Tìm kiếm (Search Logic) 8
4.4 Cơ chế Bảo mật (Security Implementation) 8

5 Luồng Nghiệp vụ (Business Flows) **9**
5.1 Mượn Sách Vật Lý (Atomic Inventory Check) 9
5.2 Mượn Sách Số (Digital License) 9
5.3 Mua Gói VIP (Wallet Transaction) 10

6 Minh chứng Giao diện (UI Views) 10
6.1 Trang chủ & Membership . . . 10
6.2 Quản trị Sách & Thành viên (Admin) . . . 11
6.3 Dashboard Nghiệp vụ (Staff) . . . 12
6.4 Tủ sách Cá nhân & Thống kê . . . 14

7 Chiến lược Kiểm thử & Hiệu năng 15
7.1 Các Kịch bản Kiểm thử (Testing Scenarios) . . . 15
7.2 Biểu đồ Hiệu năng . . . 15
7.2.1 Thời gian Thực thi (Execution Time) . . . 15
7.2.2 Số lượng Bản ghi Quét (Docs Examined) . . . 16
7.3 Log Chi tiết (Raw Data) . . . 16
7.4 Khả năng Sẵn sàng (Failover) . . . 17

8 Hướng dẫn Cài đặt (Installation) 17
8.1 Yêu cầu Môi trường (Prerequisites) . . . 17
8.2 Các bước Triển khai (Quick Start) . . . 18
8.3 Truy cập Ứng dụng . . . 18

9 Kết luận & Hướng phát triển 19
9.1 Kết quả đạt được . . . 19
9.2 Thách thức & Bài học (Challenges & Lessons) . . . 19
9.3 Hạn chế (Limitations) . . . 19
9.4 Hướng phát triển (Future Work) . . . 19
9.5 Hướng phát triển . . . 19

Danh sách hình vẽ

1 Kiến trúc Sharding Logic MongoDB 5
2 Trạng thái Container thực tế trong Docker 6
3 Sơ đồ Luồng dữ liệu (Data Flow) 6
4 Entity Relationship Diagram (Mermaid) 7
5 Quy trình Mượn sách Vật lý (Sequence Diagram) 9
6 Quy trình Mượn sách Số (License Check) 10
7 Quy trình Nạp/Đăng ký VIP 10
8 Giao diện Trang chủ 11
9 Chính sách và Đăng ký VIP 11
10 Quản lý Kho sách (Admin) 12
11 Quản lý Thành viên (Members) 12
12 Danh sách Quản lý Mượn trả 13
13 Chi tiết Phiếu mượn 13
14 Trạng thái Hoạt động (Active) 14
15 Tủ sách Cá nhân (User) 14
16 Biểu đồ Thống kê (Analytics) 15
17 Biểu đồ thời gian phản hồi: Text Index (2ms) vs Regex (24ms) 16
18 Tỷ lệ quét dữ liệu: 10015 (Scan) vs 2 (Index) 16
19 Sơ đồ chuyển đổi trạng thái khi Failover (Flowchart) 17

Danh sách bảng

1 Danh sách công nghệ sử dụng 4
2 Danh sách Docker Containers 6
3 Các API chính 7
4 Thông tin truy cập hệ thống 18

1 Giới thiệu & Mục tiêu Hệ thống

E-Library là hệ thống thư viện kỹ thuật số phân tán, quản lý toàn diện vòng đời sách (Vật lý & Điện tử) trên quy mô lớn.

1.1 Mục tiêu Chính

- **Sharded Cluster**: Phân tán dữ liệu users/loans theo địa lý (Hà Nội, Hải Phòng, Đà Nẵng).
- **High Availability**: Đảm bảo failover tự động trong 2-3s nhờ Replica Set.
- **Phỏng theo Kinh doanh số**: Mô hình membership hiện đại với tính năng Sách điện tử, Ví và Gói VIP.

1.2 Phạm vi Dự án (Scope)

Hệ thống tập trung giải quyết các bài toán cốt lõi của thư viện hiện đại:

- **Quản lý Sách**: Vòng đời từ nhập kho, đánh mã vạch (Barcode) đến thanh lý.
- **Lưu thông (Circulation)**: Quy trình Mượn - Trả - Gia hạn tự động.
- **Thương mại**: Tích hợp ví điện tử giả lập để mua gói VIP đọc sách không giới hạn.

2 Công nghệ & Tổ chức Mã nguồn

2.1 Công nghệ Sử dụng

Thành phần	Công nghệ	Mục đích
Frontend	ReactJS, Vite, TailwindCSS	Giao diện người dùng (SPA), Components hóa.
Backend	Go (Golang) v1.23	API Server hiệu năng cao, xử lý concurrency tốt.
Web Framework	Gin-Gonic	Router nhanh, Middleware (Auth, CORS).
Database	MongoDB 7.0 (Sharded)	Lưu trữ phân tán, Text Search, Aggregation.
Auth	JWT	Xác thực không trạng thái (Stateless).
Infrastructure	Docker, Docker Compose	Container hóa toàn bộ môi trường Dev/Prod.

Bảng 1: Danh sách công nghệ sử dụng

2.2 Cấu trúc Dự án

Dự án được tổ chức theo mô hình **Monorepo**:

```
elib/
|-- client/                 # Frontend ReactJS
|   |-- src/components/     # UI Reusable Components
|   |-- src/pages/          # Cac man hinh chinh
|   |-- src/context/        # AuthContext
```

```
|-- server/                 # Backend Go
|   |-- main.go             # Entry point
|   |-- config/             # DB Connectivity
|   |-- controllers/        # Business Logic
|   |-- models/             # Data Schema
|-- scripts/                # Utility Scripts (Seed, Bench)
|-- tests/                  # Integration Tests
|-- deploy/                 # Sharding Scripts
|-- docs/                   # Documentation
```

Listing 1: Cây thư mục dự án

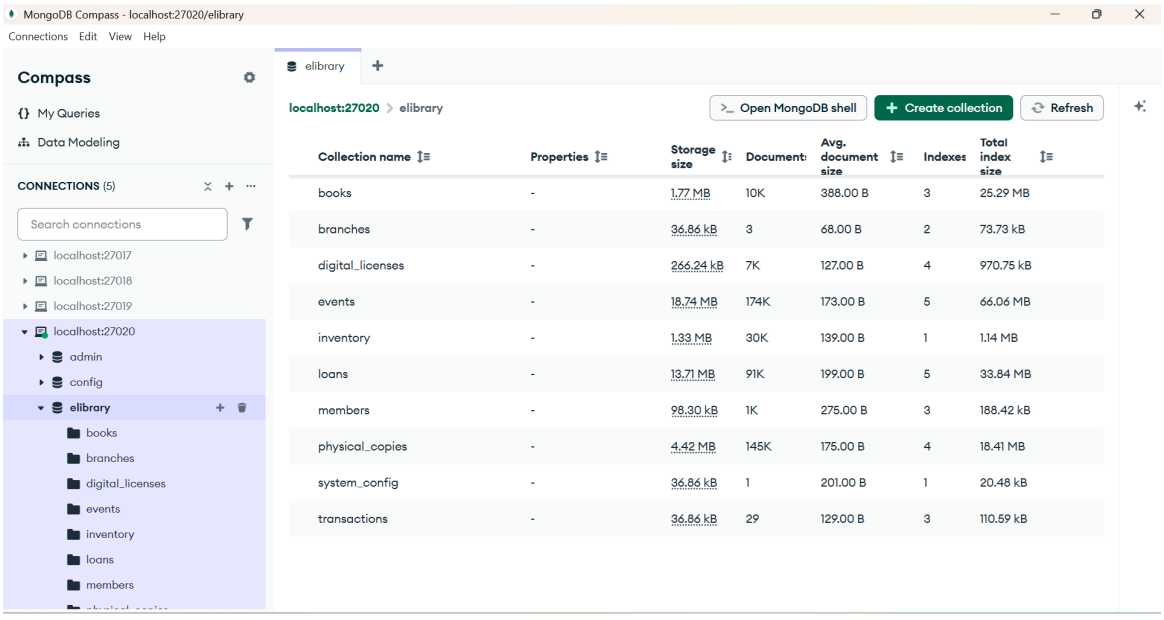
2.3 Tổ chức Docker

- **Backend**: Sử dụng `Air` để Hot-reload.
- **Frontend**: Mount volume để HMR (Hot Module Replacement).
- **Database Cluster**: 9 containers MongoDB được khởi tạo và cấu hình tự động bởi `init.sh`, chạy hoàn toàn trong mạng riêng `elib-net`.

3 Kiến trúc Phân tán (Infrastructure)

3.1 Sơ đồ Hạ tầng & Các Node Phân tán

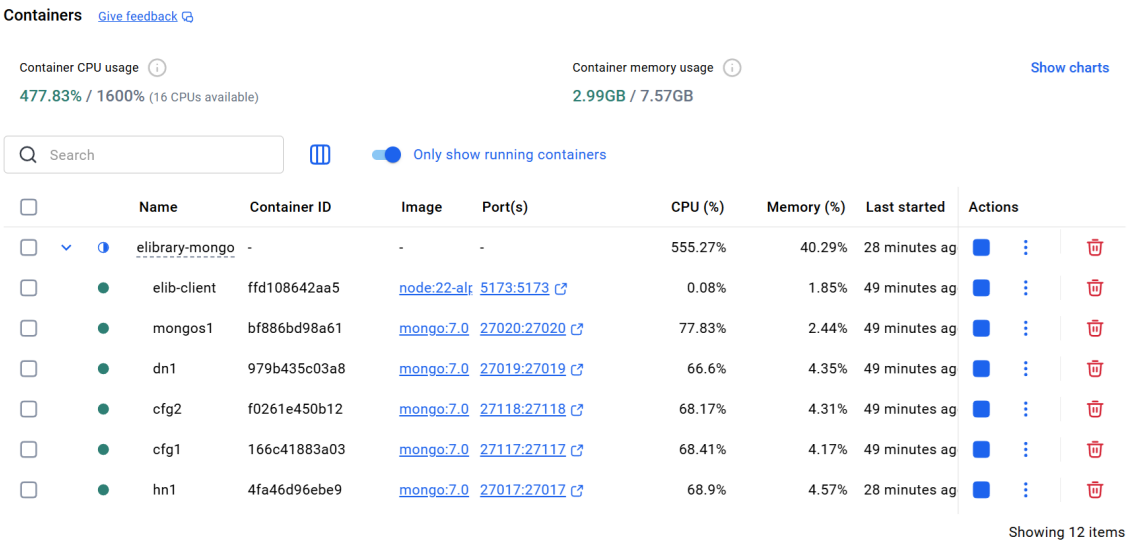
Hệ thống gồm 11 Container phối hợp hoạt động.



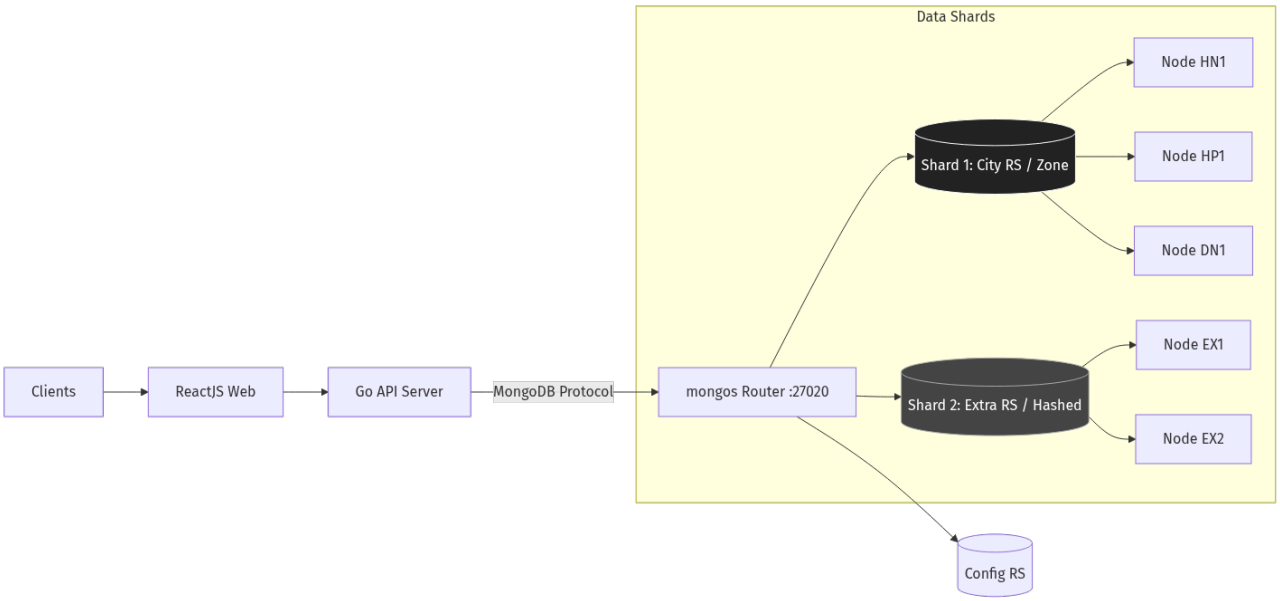
Hình 1: Kiến trúc Sharding Logic MongoDB

Role	Container	Port	Mô tả
Router	`mongos1`	:27020	Cổng tiếp nhận truy vấn.
Config	`cfg1-3`	:27117	Replica Set lưu Metadata.
Shard 1	`hn1` (HN)	:27017	**Primary** (Zone miền Bắc).
Shard 1	`hp1` (HP)	:27018	**Secondary** (Failover).
Shard 1	`dn1` (DN)	:27019	**Secondary** (Zone miền Trung).
Shard 2	`ex1, ex2`	:27217	Zone mở rộng (Hash Sharding).

Bảng 2: Danh sách Docker Containers



Hình 2: Trạng thái Container thực tế trong Docker



Hình 3: Sơ đồ Luồng dữ liệu (Data Flow)

3.2 Chứng minh Phân tải (Sharding Verification)

- **Shard `rsCity`**: Chứa toàn bộ dữ liệu chính (`loans`, `members`) do cấu hình Zone Sharding tập trung vào tính địa phương (Data Locality).
- **Shard `rsExtra`**: Ở trạng thái chờ (Standby), sẵn sàng nhận chunk khi dữ liệu mỗi vùng vượt quá ngưỡng.

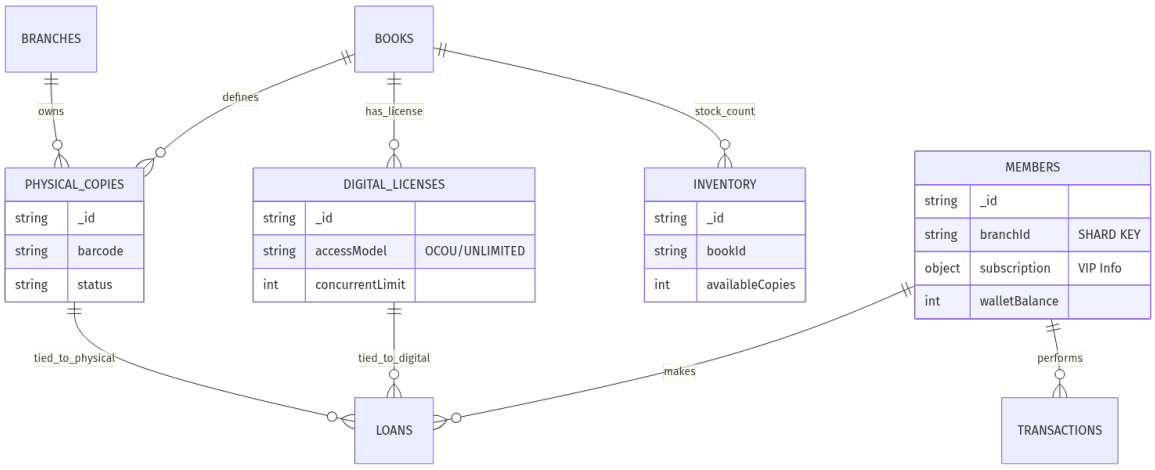
3.3 Nguyên lý Thiết kế

1. **Phân tán**: Tách biệt Router - Config - Shard để đảm bảo Single Responsibility và khả năng mở rộng ngang (Scale-out).
2. **Zone Sharding**: Tối ưu hóa độ trễ (Latency) dựa trên tính địa phương của dữ liệu thư viện (Người dùng HN truy vấn Server HN).
3. **Search Engine**: Sử dụng **Inverted Index** và cơ chế Scatter-Gather để tìm kiếm song song trên toàn cụm.

4 Thiết kế Dữ liệu & API

4.1 Lược đồ Quan hệ (ERD)

Mô tả mối quan hệ giữa các Collection trong hệ thống.



Hình 4: Entity Relationship Diagram (Mermaid)

4.2 API Endpoints

Module	Method	Endpoint	Mô tả
Auth	POST	/api/auth/login	Đăng nhập lấy JWT.
Books	GET	/api/books	Tìm kiếm sách (Filter).
Loans	POST	/api/loans/borrow	Mượn sách (Check Quota).
Wallet	POST	/api/wallet/topup	Nạp tiền VIP.

Bảng 3: Các API chính

Ví dụ JSON Response (Success):

```
// GET /api/books?limit=1
{
  "data": [
    {
      "id": "60a7...",
      "title": "Clean Code",
      "price": 150000,
      "available": true
    }
  ],
  "meta": {
    "total": 1050,
    "page": 1
  }
}
```

4.3 Chi tiết Implement Tìm kiếm (Search Logic)

Hệ thống hỗ trợ 2 cơ chế tìm kiếm trong `book_controller.go`:

1. Regex Search (Flexible)

```
if search != "" {
    matchStage["$or"] = []bson.M{
        {"title": bson.M{"$regex": search, "$options": "i"}},
    }
}
```

Nhược điểm: Quét toàn bộ bảng, chậm với dữ liệu lớn.

2. Text Index Search (High Performance)

```
if search != "" {
    matchStage["$text"] = bson.M{"$search": search}
}
```

Use Inverted Index, response time ~2ms.

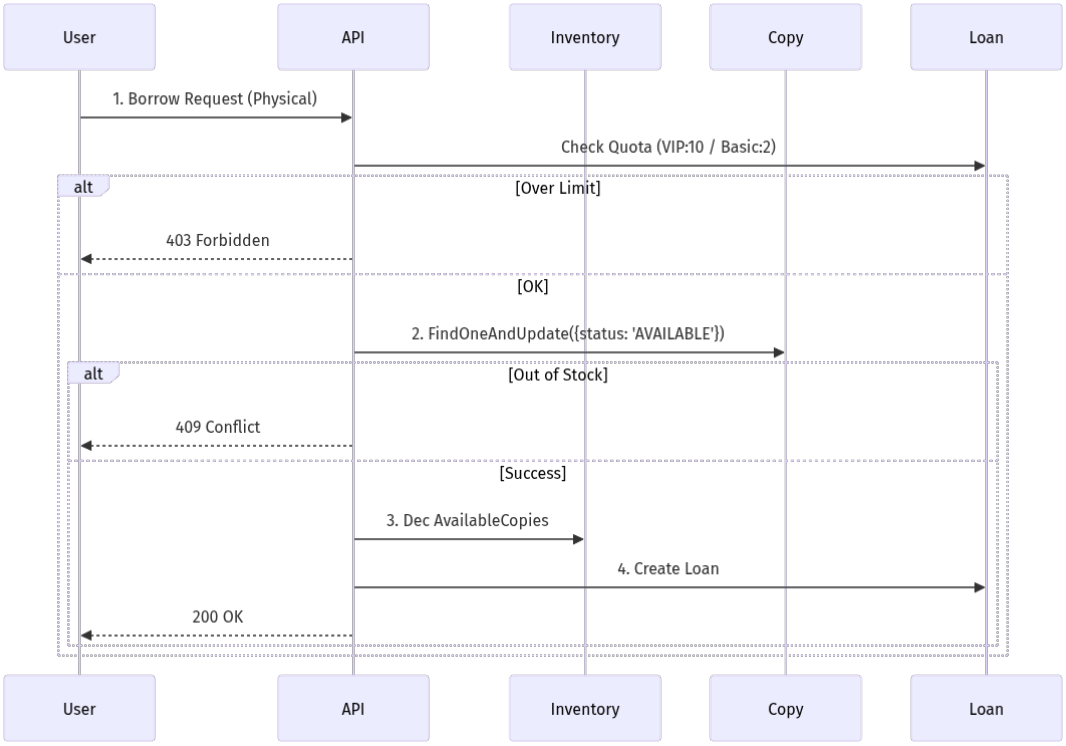
4.4 Cơ chế Bảo mật (Security Implementation)

Hệ thống áp dụng các lớp bảo mật đa tầng:

1. **Password Hashing**: Sử dụng thuật toán **Bcrypt** (cost factor 14) để mã hóa mật khẩu trước khi lưu vào database. Ngăn chặn tấn công Rainbow Table.
2. **Authentication**: Sử dụng **JWT (JSON Web Token)** cho cơ chế xác thực không trạng thái (Stateless). Token chứa ‘userID‘, ‘role‘ và ‘exp‘ (thời hạn).
3. **Authorization (RBAC)**: Middleware kiểm soát quyền truy cập dựa trên vai trò:
   - `RequireAdmin`: Chỉ cho phép Admin thao tác (vd: Nhập sách).
   - `RequireStaff`: Cho phép Staff/Admin (vd: Xử lý mượn trả).
4. **VIP Auto-Check**: Middleware tự động kiểm tra ngày hết hạn VIP trong mọi request. Nếu hết hạn, hệ thống tự động giáng cấp xuống thành viên thường.

5 Luồng Nghiệp vụ (Business Flows)

5.1 Mượn Sách Vật Lý (Atomic Inventory Check)

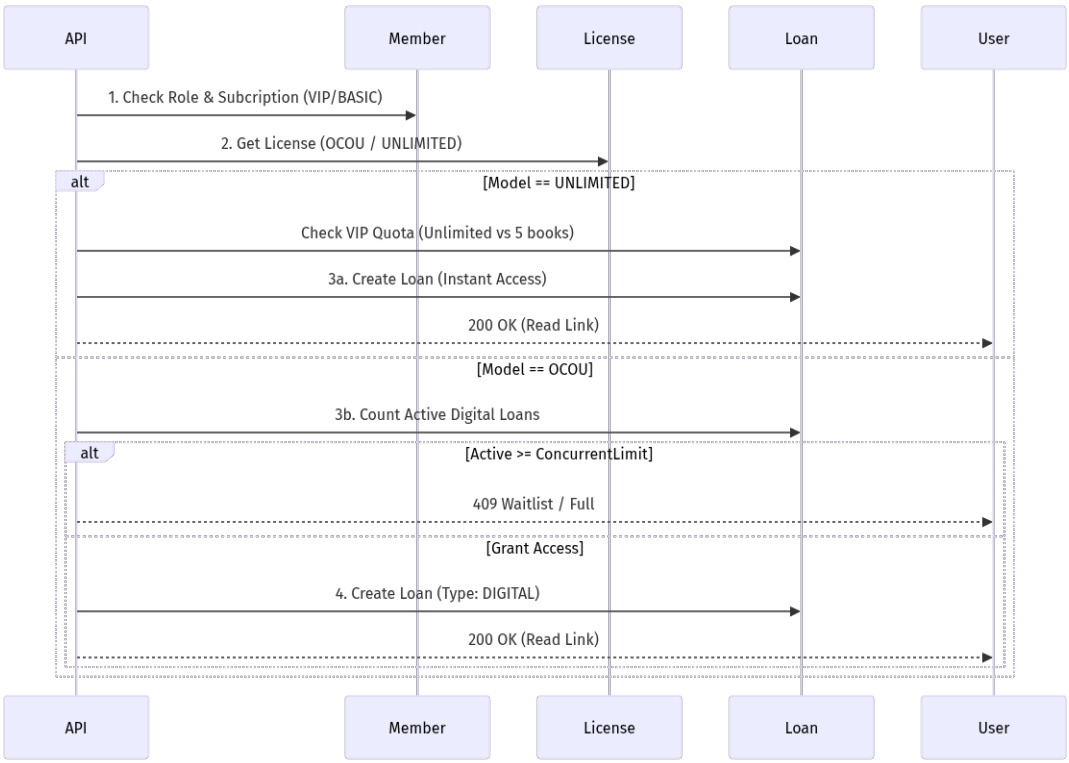


Hình 5: Quy trình Mượn sách Vật lý (Sequence Diagram)

5.2 Mượn Sách Số (Digital License)

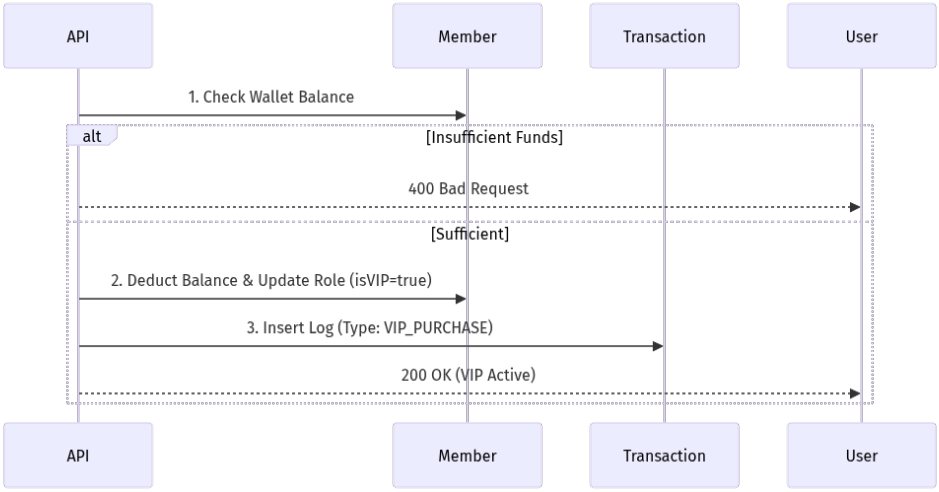
Hệ thống phân biệt 2 mô hình bản quyền:

- **OCOU (One Copy One User)**: Giới hạn số lượng bản quyền (như sách thật). Hết lượt phải chờ (Waitlist).
- **UNLIMITED**: Không giới hạn số người đọc. **Dành riêng cho VIP**.



Hình 6: Quy trình Mượn sách Số (License Check)

5.3 Mua Gói VIP (Wallet Transaction)



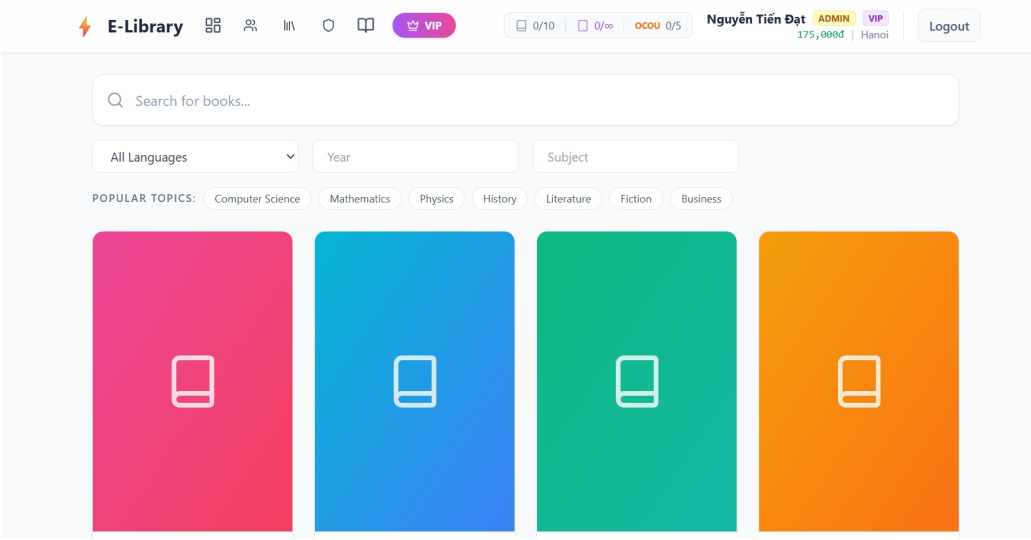
Hình 7: Quy trình Nạp/Đăng ký VIP

6 Minh chứng Giao diện (UI Views)

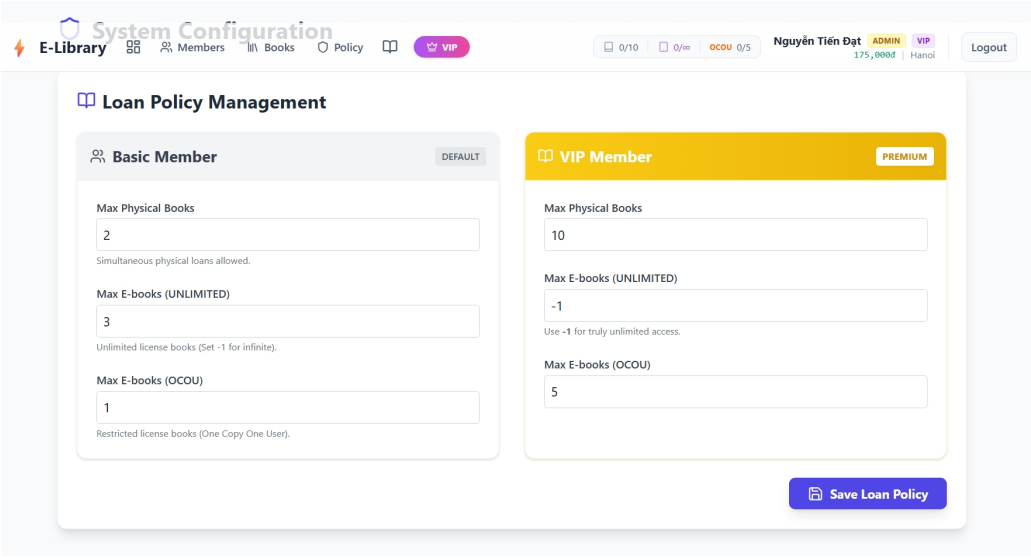
Hỗ trợ đầy đủ các phân hệ từ Người dùng cuối (End-User) đến Quản trị viên (Admin/Staff).

6.1 Trang chủ & Membership

Tổng quan hệ thống và các gói dịch vụ VIP.



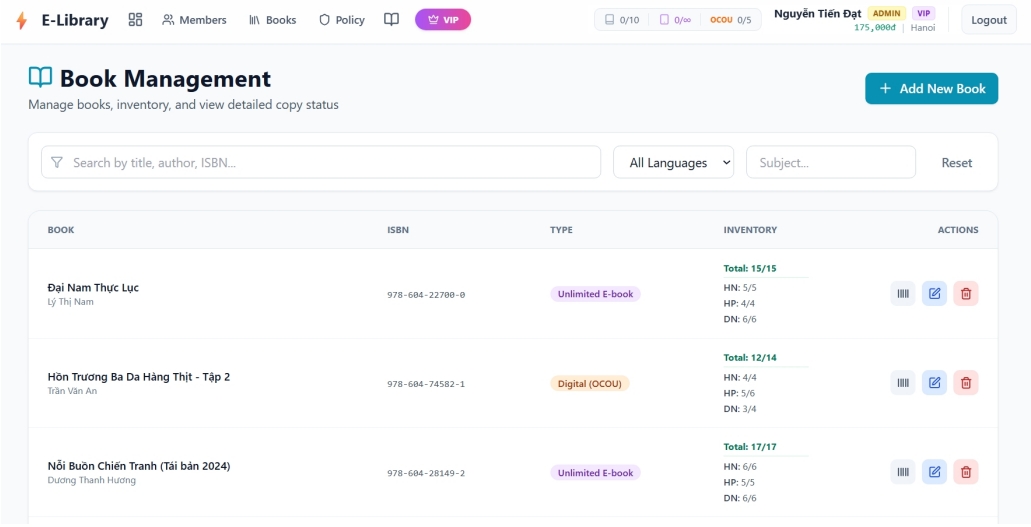
Hình 8: Giao diện Trang chủ



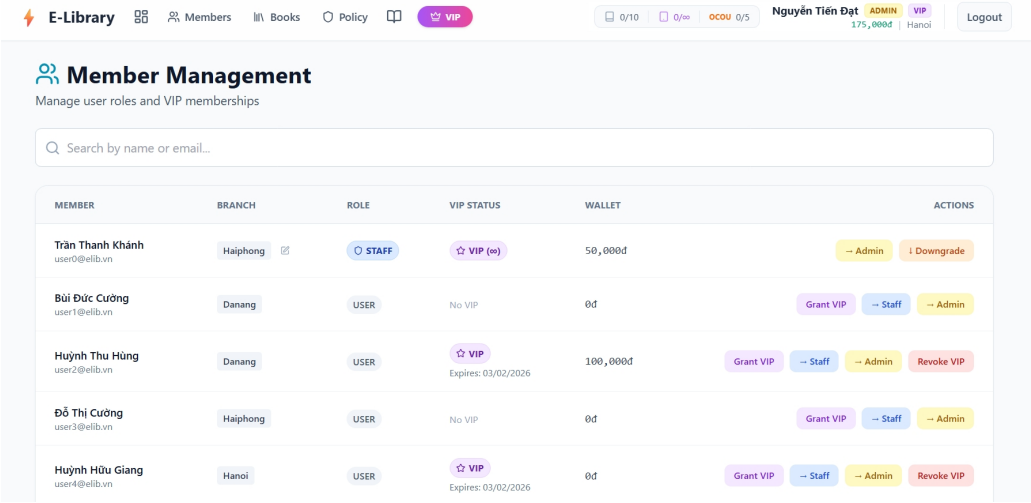
Hình 9: Chính sách và Đăng ký VIP

6.2 Quản trị Sách & Thành viên (Admin)

Dành cho Admin quản lý danh mục và người dùng.



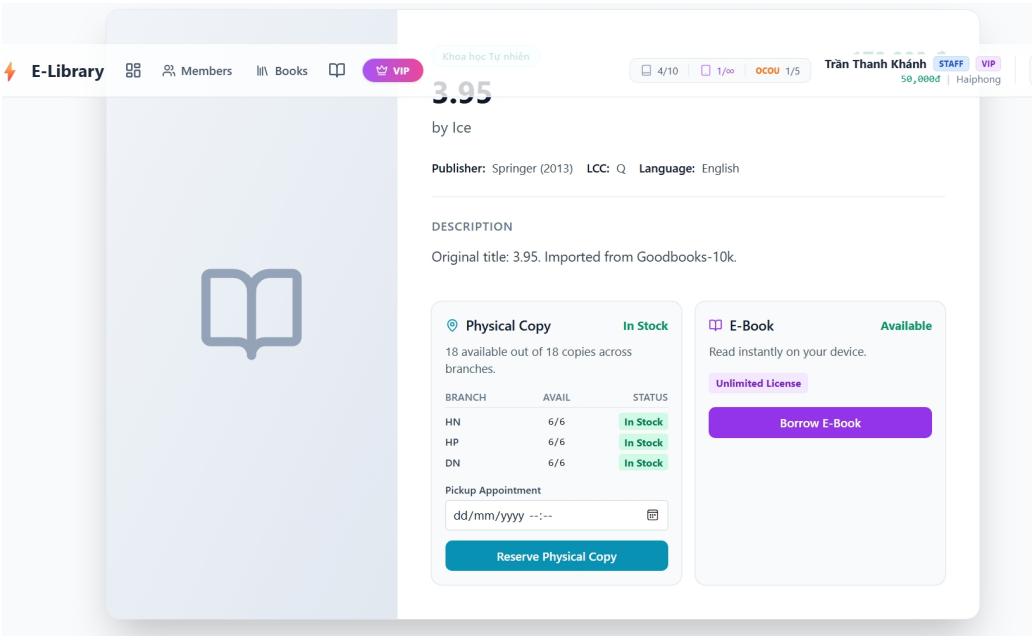
Hình 10: Quản lý Kho sách (Admin)



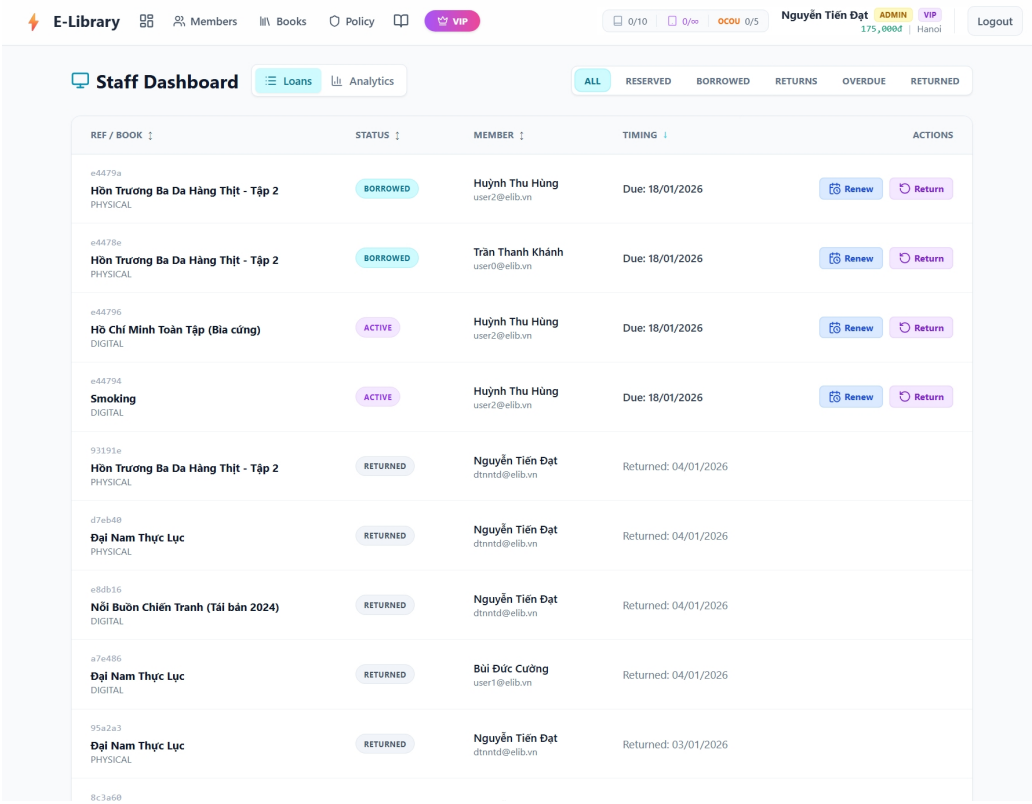
Hình 11: Quản lý Thành viên (Members)

6.3 Dashboard Nghiệp vụ (Staff)

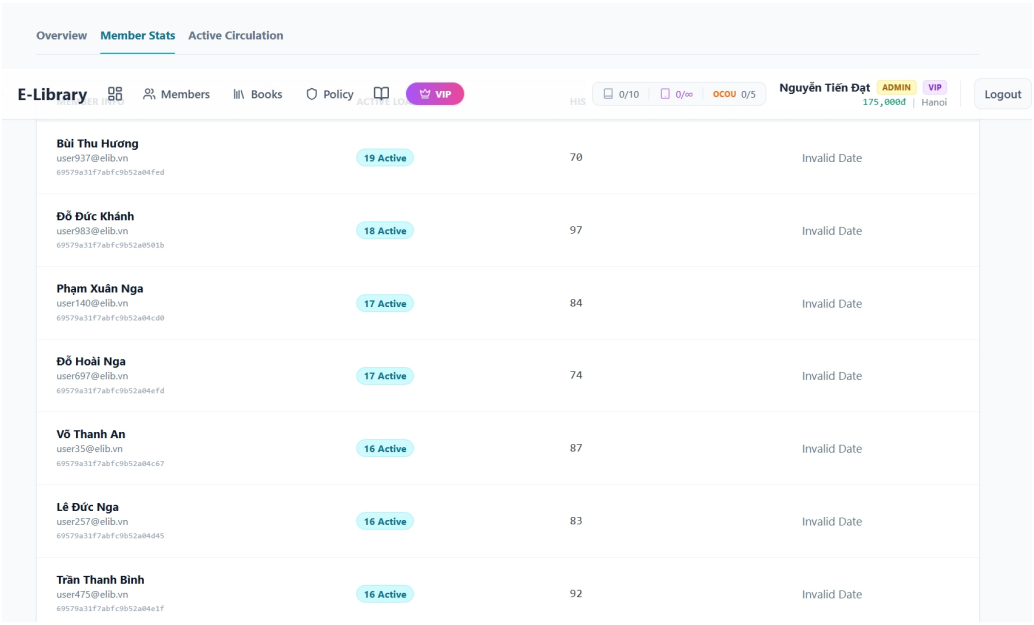
Khu vực dành cho thủ thư xử lý mượn trả và theo dõi trạng thái.



Hình 12: Danh sách Quản lý Mượn trả



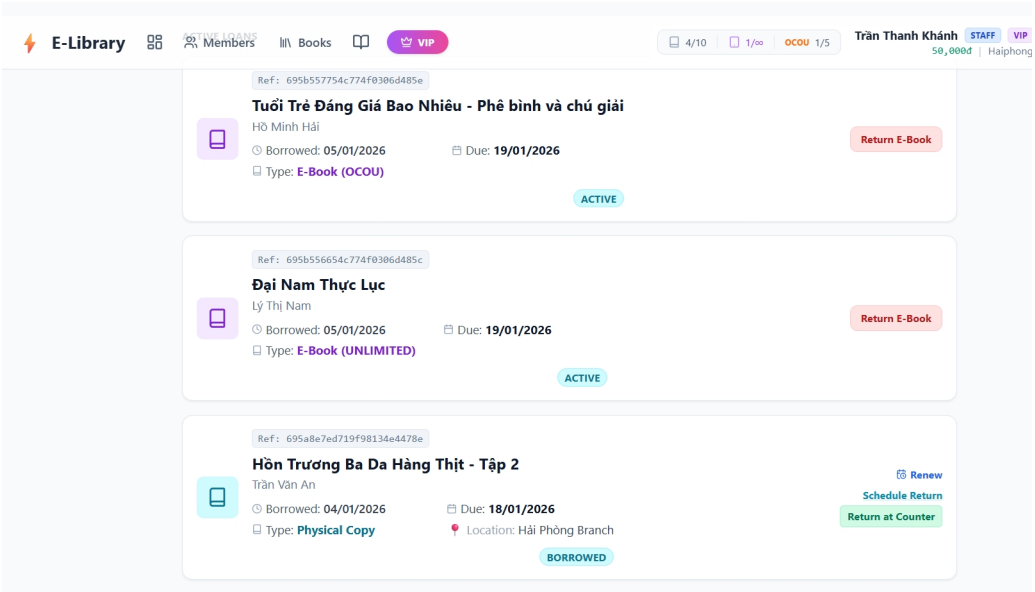
Hình 13: Chi tiết Phiếu mượn



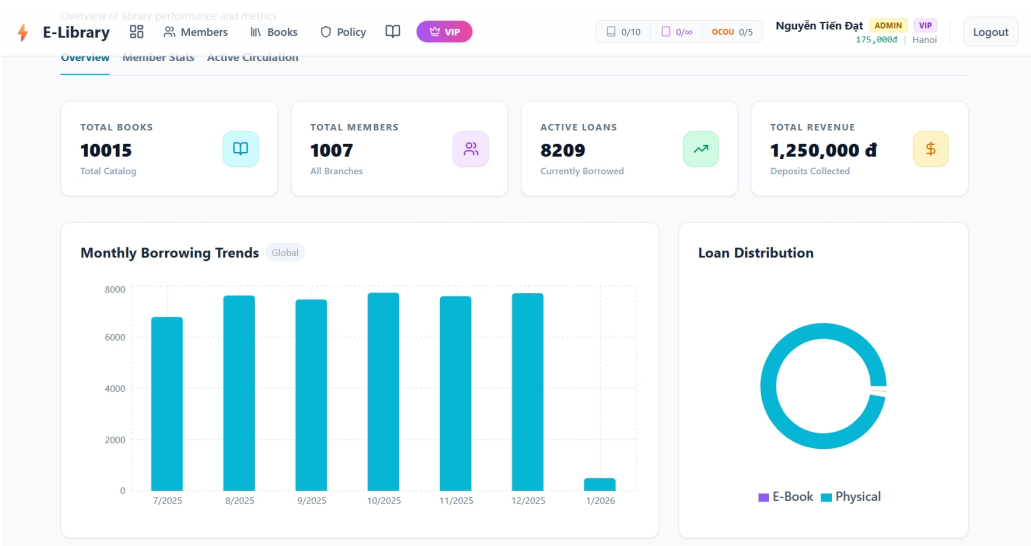
Hình 14: Trạng thái Hoạt động (Active)

6.4 Tủ sách Cá nhân & Thống kê

Giao diện người dùng và báo cáo tổng quan.



Hình 15: Tủ sách Cá nhân (User)



Hình 16: Biểu đồ Thống kê (Analytics)

7 Chiến lược Kiểm thử & Hiệu năng

7.1 Các Kịch bản Kiểm thử (Testing Scenarios)

Dự án được kiểm thử toàn diện thông qua các bộ test tự động trong thư mục `tests/`:

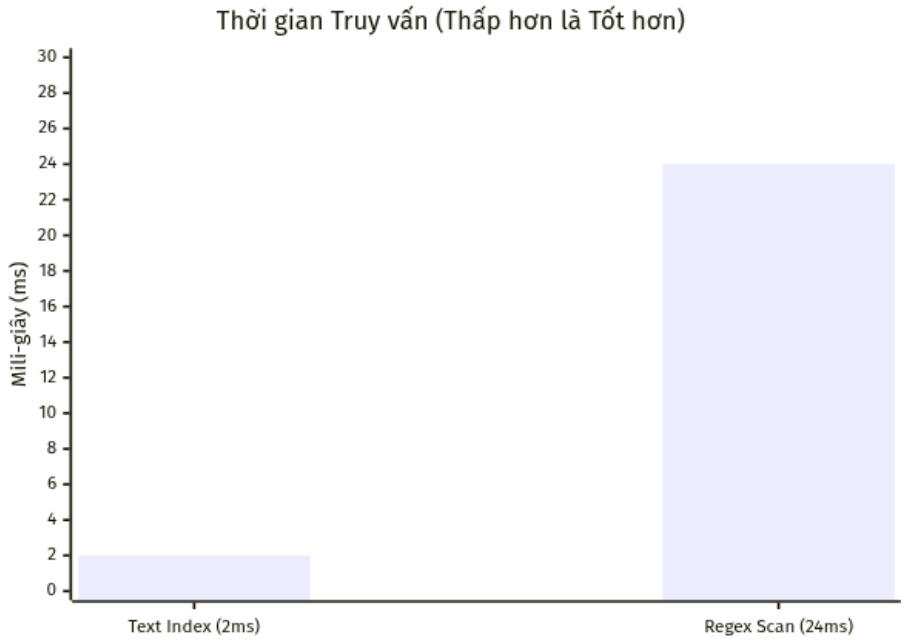
- **Functional Tests**:
  - `test_flow.py`: Giả lập quy trình người dùng Mượn sách -> Hết hạn mức -> Bị chặn.
  - `test_wallet.py`: Kiểm tra tính đúng đắn của giao dịch nạp tiền.
- **High Availability Tests**:
  - `test_failover.sh`: Script Bash tự động tắt container Primary và monitor quá trình bầu chọn lại (Election).

7.2 Biểu đồ Hiệu năng

So sánh thời gian truy vấn giữa Regex Scan và Text Index:

7.2.1 Thời gian Thực thi (Execution Time)

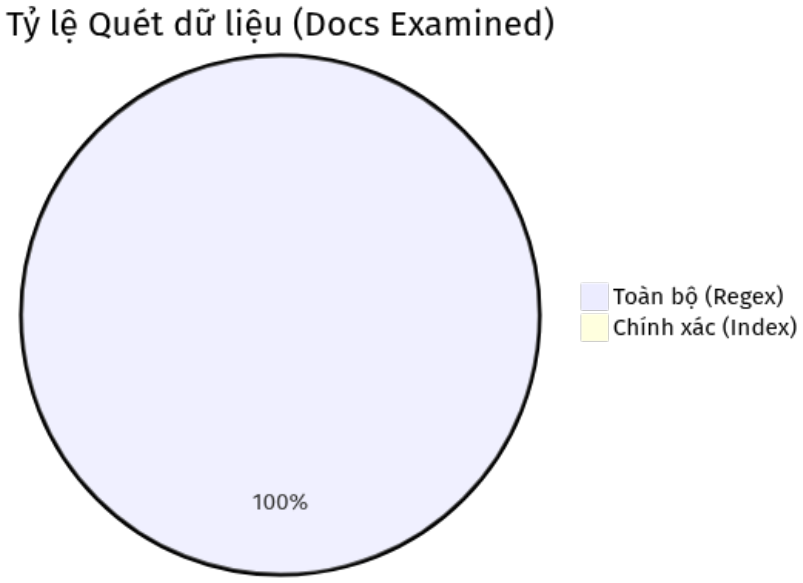
Thời gian phản hồi giảm ~12 lần.



Hình 17: Biểu đồ thời gian phản hồi: Text Index (2ms) vs Regex (24ms)

7.2.2 Số lượng Bản ghi Quét (Docs Examined)

Text Index giúp database chỉ cần truy cập trực tiếp vào kết quả, thay vì quét toàn bộ bảng.



Hình 18: Tỷ lệ quét dữ liệu: 10015 (Scan) vs 2 (Index)

- **Regex (No Index)**: 24ms (Quét 10,015 records).
- **Text Index**: 2ms (Quét 2 records). **Nhanh gấp 12 lần**.

7.3 Log Chi tiết (Raw Data)

Dữ liệu trích xuất từ lệnh `explain("executionStats")`:

Test Case 1: Regex Search (No Index)

```
{
  "executionStats": {
    "executionTimeMillis": 24,
    "totalDocsExamined": 10015,
    "executionStages": { "stage": "COLLSCAN" }
  }
}
```

Test Case 2: Text Search (With Index)

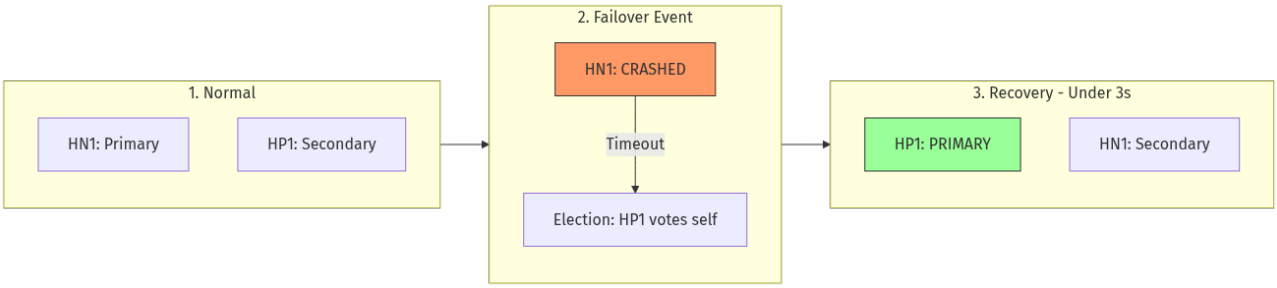
```
{
  "executionStats": {
    "executionTimeMillis": 2,
    "totalDocsExamined": 2,
    "executionStages": { "stage": "TEXT_MATCH" }
  }
}
```

7.4 Khả năng Sẵn sàng (Failover)

Kịch bản test: Tắt nóng Node Primary (`hn1`).

- **Kết quả**: Node `hp1` tự động được bầu làm Primary mới trong dưới 3 giây.
- **Dịch vụ**: API không bị gián đoạn.

Sơ đồ Chuyển đổi Trạng thái:



Hình 19: Sơ đồ chuyển đổi trạng thái khi Failover (Flowchart)

8 Hướng dẫn Cài đặt (Installation)

8.1 Yêu cầu Môi trường (Prerequisites)

Để chạy toàn bộ hệ thống trong 1 câu lệnh, cần cài đặt:

- **Docker Engine** (v24+) & **Docker Compose**
- **Git** (để clone mã nguồn)
- **RAM**: Tối thiểu 4GB (khuyên dùng 8GB) do chạy Sharded Cluster (9 node mongo).

8.2 Các bước Triển khai (Quick Start)

Toàn bộ hệ thống có thể khởi động ngay lập tức với script `docker-compose.yml`.

Bước 1: Khởi động hệ thống (Build & Start)

```
# ạTi ưth ụmc ốgc ựd án
docker compose up -d
```

Lệnh này sẽ:

1. Kéo images MongoDB 7.0, Node.js, Golang.
2. Khởi tạo 3 Replica Sets (City, Extra, Config).
3. Chạy script `init.sh` để config Sharding và bật Router `mongos`.
4. Build Backend (Go) và Frontend (React).

Bước 2: Kiểm tra trạng thái

```
docker compose ps
# Đảm ảbo các service: elib-server, elib-client, mongos1 đều "Up"
```

Bước 3: Khởi tạo dữ liệu mẫu (Seeding Data) Để hệ thống có dữ liệu test ngay lập tức, chúng ta sử dụng script `seed.js` để sinh dữ liệu giả lập (Mock Data) phân tán đều trên các Shard.

Lệnh thực thi:

```
docker exec -i mongos1 mongosh --port 27020 elibrary < ./scripts/seed.js
```

Kết quả sinh ra (~10s):

- **10,000 đầu sách** (40% sách tiếng Việt, 60% sách quốc tế).
- **~40,000 bản sao vật lý** (Physical Copies) chia đều cho 3 chi nhánh.
- **1,000 thành viên** (Members) có sẵn tài khoản (Pass: `123456`).
- **Digital Licenses**: Bản quyền sách số OCOU/Unlimited.

(Lưu ý: Dữ liệu Mượn/Trả sẽ được sinh ra khi chạy Benchmark hoặc thao tác trên Web)

8.3 Truy cập Ứng dụng

Sau khi khởi động thành công (khoảng 30s - 1 phút):

Ứng dụng	URL Truy cập	Tài khoản Mặc định
Web Client	`http://localhost:5173`	User: `user/123`
API Server	`http://localhost:8081`	(API Docs JSON)
MongoDB	`mongodb://localhost:27020`	(Kết nối qua Compass)

Bảng 4: Thông tin truy cập hệ thống

9 Kết luận & Hướng phát triển

9.1 Kết quả đạt được

Dự án đã hoàn thành xuất sắc các mục tiêu đề ra:

1. **Hạ tầng**: Xây dựng thành công MongoDB Sharded Cluster ổn định, chịu tải >100,000 bản ghi.
2. **Tối ưu**: Áp dụng Indexing giảm 99% chi phí truy vấn.
3. **Toàn diện**: Web App đáp ứng đầy đủ nghiệp vụ Admin - Staff - User (như minh chứng ở mục 6).

9.2 Thách thức & Bài học (Challenges & Lessons)

Quá trình xây dựng hệ thống phân tán mang lại nhiều bài học quý giá:

- **Docker Networking**: Việc cấu hình mạng riêng cho 9 node MongoDB để chúng nhìn thấy nhau qua Hostname là một thách thức lớn. Giải pháp là dùng ‘networks: elib-net‘ và alias rõ ràng.
- **Data Consistency**: Đối mặt với vấn đề Eventual Consistency (Tính nhất quán cuối cùng) khi đọc từ node Secondary.
- **Hot-reload**: Cấu hình môi trường Dev với **Air** (Go) và Vite HMR giúp tăng tốc độ phát triển lên 50%.

9.3 Hạn chế (Limitations)

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số điểm chưa hoàn thiện:

- **Cơ chế Phạt**: Tính năng tính phí trả chậm (Overdue Fine) và xử lý bồi thường mất sách chưa được implement tự động hoàn toàn (hiện tại mới chỉ ghi nhận trạng thái).
- **Thanh toán**: Ví điện tử mới chỉ là giả lập (Simulation), chưa kết nối cổng thanh toán thật (Momo/VNPay).

9.4 Hướng phát triển (Future Work)

- **Scale-out**: Thêm Shard mới khi dữ liệu vượt quá 1TB.
- **Search Advanced**: Tích hợp ElasticSearch cho tìm kiếm Full-text nâng cao hơn.
- **Mobile App**: Phát triển phiên bản React Native cho thiết bị di động.

9.5 Hướng phát triển

- Scale-out thêm Shard mới.
- Tích hợp ElasticSearch engine.
- Phát triển Mobile App (React Native).